西安交通大学实验报告

课程 Rust 程序设计 实验名称__实验二____ 第 1 页 共 15 页

系 别_____计算机科学与技术_____ 实 验 日 期 2026 年 4 月 22 日
专业班级_____组别_____6_____ 实 验 报 告 日 期 2026 年 4 月 22 日
姓 名_____学号 报 告 退 发（订正、重做）
同 组 人____、、（该实验独立完成）_____
教 师 审 批 签 字

实验名称：实验二

一. 题目

任务一：使用 hyper 或 actix-web 库编写一个简单的 HTTP 服务器，当访问/hello 路径时返回"Hello, Rust!"文本响应。

任务二：扩展基础 HTTP 服务器，使其能托管静态文件（如 HTML/CSS），当访问根路径/时返回一个自定义的欢迎页面。

任务三：用 serde 和 actix-web 实现一个图书管理 API，支持 GET /books（获取所有书籍）和 POST /books（添加新书籍），数据暂存内存中。

任务四：为 Web 服务器添加日志中间件，记录每个请求的路径、响应状态码和耗时，使用 tracing 或 log 库输出到控制台。

二. 环境

- 操作系统：Ubuntu 24.04.3 LTS
- 运行平台：WSL2
- 内核版本：Linux 6.6.87.2-microsoft-standard-WSL2
- 处理器：13th Gen Intel(R) Core(TM) i5-13500HX
- CPU 核心：4 逻辑 CPU

- 内存：7.8 GiB
- 交换空间：4.0 GiB
- Rust 工具链：stable-x86_64-unknown-linux-gnu
- 编译器版本：rustc 1.94.0
- 构建工具版本：cargo 1.94.0

三. 实验步骤

任务一：简单 HTTP 服务器代码：

```
use actix_web::{web, App, HttpResponse, HttpServer, Responder};

async fn hello() -> impl Responder {

   HttpResponse::Ok().body("Hello, Rust!")

}

#[actix_web::main]

async fn main() -> std::io::Result<()> {

   println!("Server running at http://127.0.0.1:8080");



   HttpServer::new(|| {

      App::new()

         .route("/hello", web::get().to(hello))

   })

   .bind(("127.0.0.1", 8080))?

   .run()
```

```
    .await
}
```

分析：使用 actix-web 框架搭建最基础的 HTTP 服务器。定义异步处理函数 hello()，返回类型为 impl Responder，通过 HttpResponse::Ok().body("Hello, Rust!")构造一个状态码为 200 的纯文本响应。main 函数以 #[actix_web::main]宏标注，将其转换为异步运行时入口。HttpServer::new 接收一个闭包，闭包内通过 App::new()创建应用实例，并用.route("/hello", web::get().to(hello))将 GET /hello 路径绑定到 hello 处理函数。最后调用.bind()绑定地址和端口，.run().await 启动事件循环。

任务二：欢迎页面

html 文件：

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <title>Rust Lab 2</title>
</head>
<body>
    <h1>欢迎来到 Rust Lab 2</h1>
    <p>这是任务二的欢迎页面。</p>
    <p>可以访问 <a href="/hello">/hello</a> 查看文本响应。</p>
```

```
</body>

</html>
```

main.rs:

```
use actix_files::Files;

use actix_web::{web, App, HttpResponse, HttpServer, Responder};

async fn hello() -> impl Responder {

    HttpResponse::Ok().body("Hello, Rust!")

}

#[actix_web::main]

async fn main() -> std::io::Result<()> {

    println!("Server running at http://127.0.0.1:8080");



    HttpServer::new(|| {

        App::new()

            .route("/hello", web::get().to(hello))

            .service(

                Files::new("/", "./static")

                    .index_file("index.html")

            )

    })

    .bind(("127.0.0.1", 8080))?
```

```
    .run()
    .await
}
```

分析：在任务一的基础上引入 actix-files 库实现静态文件托管。HTML 文件放置在项目根目录下的 static/文件夹中， index.html 作为默认首页。代码中通过 Files::new("/", "./static").index_file("index.html")将根路径 /映射到 static 目录，并指定 index.html 为索引文件。需要注意.service(Files::new(...)) 应放在.route("/hello", ...) 之后，因为 actix-web 按注册顺序匹配路由，精确路由优先于文件服务的通配匹配，这样访问/hello 时仍然返回文本响应，访问/时则返回欢迎页面。

任务三：图书管理 API

```
use actix_files::Files;
use actix_web::{web, App, HttpResponse, HttpServer, Responder};
use serde::{Deserialize, Serialize};
use std::sync::Mutex;


#[derive(Serialize, Deserialize, Clone)]
struct Book {
    id: u32,
    title: String,
    author: String,
}
```

```
struct AppState {
    books: Mutex<Vec<Book>>,
}

async fn hello() -> impl Responder {
    HttpResponse::Ok().body("Hello, Rust!")
}

async fn get_books(data: web::Data<AppState>) -> impl Responder {
    let books = data.books.lock().unwrap();
    HttpResponse::Ok().json(books.clone())
}

async fn add_book(data: web::Data<AppState>, new_book: web::Json<Book>) -> impl
Responder {
    let mut books = data.books.lock().unwrap();
    let book = new_book.into_inner();
    books.push(book.clone());
    HttpResponse::Created().json(book)
}
```

```rust
#[actix_web::main]

async fn main() -> std::io::Result<()> {

    let state = web::Data::new(AppState {

        books: Mutex::new(vec![

            Book {

                id: 1,

                title: "The Rust Programming Language".to_string(),

                author: "Steve Klabnik & Carol Nichols".to_string(),

            }

        ]),

    });



    println!("Server running at http://127.0.0.1:8080");



    HttpServer::new(move || {

        App::new()

            .app_data(state.clone())

            .route("/hello", web::get().to(hello))

            .route("/books", web::get().to(get_books))

            .route("/books", web::post().to(add_book))

            .service(Files::new("/", "./static").index_file("index.html"))

    })
```

```
    .bind(("127.0.0.1", 8080))?
    .run()
    .await
}
```

分析：本任务引入 serde 库实现 JSON 序列化与反序列化，定义 Book 结构体并通过 #[derive(Serialize, Deserialize, Clone)] 派生宏自动生成序列化/反序列化实现。为了在多个请求之间共享可变数据，定义 AppState 结构体，内部使用 Mutex<Vec<Book>> 包裹书籍列表——Mutex 保证并发安全，web::Data 则是 actix-web 提供的应用级共享状态容器，底层基于 Arc 实现跨线程共享。

get_books 处理函数通过 data.books.lock().unwrap() 获取互斥锁，将书籍列表以 JSON 格式返回；add_book 函数从请求体中反序列化出 Book 对象，将其追加到列表中，并以 201 Created 状态码返回新增的书籍。main 函数中预置了一本书作为初始数据，闭包使用 move 关键字捕获 state 的所有权，每次创建 worker 时通过 state.clone() 共享同一份 Arc 指针。

任务四：日志输出

```
use actix_files::Files;
use actix_web::middleware::Logger;
use actix_web::{web, App, HttpResponse, HttpServer, Responder};
use env_logger::Env;
use serde::{Deserialize, Serialize};
use std::sync::Mutex;
```

```rust
#[derive(Serialize, Deserialize, Clone)]
struct Book {
    id: u32,
    title: String,
    author: String,
}

struct AppState {
    books: Mutex<Vec<Book>>,
}

async fn hello() -> impl Responder {
    HttpResponse::Ok().body("Hello, Rust!")
}

async fn get_books(data: web::Data<AppState>) -> impl Responder {
    let books = data.books.lock().unwrap();
    HttpResponse::Ok().json(books.clone())
}
```

```
async fn add_book(data: web::Data<AppState>, new_book: web::Json<Book>) -> impl

Responder {

    let mut books = data.books.lock().unwrap();

    let book = new_book.into_inner();

    books.push(book.clone());

    HttpResponse::Created().json(book)

}



#[actix_web::main]

async fn main() -> std::io::Result<()> {

    env_logger::init_from_env(Env::default().default_filter_or("info"));



    let state = web::Data::new(AppState {

        books: Mutex::new(vec![

            Book {

                id: 1,

                title: "The Rust Programming Language".to_string(),

                author: "Steve Klabnik & Carol Nichols".to_string(),

            }

        ]),

    });
```

```
    println!("Server running at http://127.0.0.1:8080");

    HttpServer::new(move || {
        App::new()
            .wrap(Logger::new("%r %s %D ms"))
            .app_data(state.clone())
            .route("/hello", web::get().to(hello))
            .route("/books", web::get().to(get_books))
            .route("/books", web::post().to(add_book))
            .service(Files::new("/", "./static").index_file("index.html"))
    })
    .bind(("127.0.0.1", 8080))?
    .run()
    .await
}
```

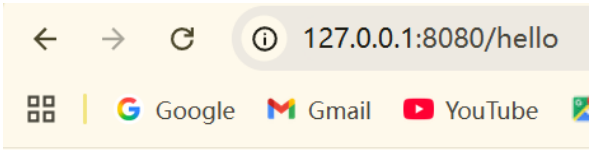
分析：actix-web 内置了 Logger 中间件，通过 .wrap(Logger::new(...)) 将其注册到应用中。格式字符串 "%r %s %D ms" 中，%r 表示请求行，%s 表示响应状态码，%D 表示请求处理耗时（毫秒）。Logger 底层依赖 log crate 的日志接口，因此需要一个具体的日志实现来将日志输出到控制台，这里选用 (env_logger)，通过 env_logger::init_from_env(Env::default().default_filter_or("info")) 初始化，默认日志级别为 info，可通过环境变量 RUST_LOG 动态调整。每当有请求到达时，中

间件会自动记录请求路径、响应状态码和处理耗时，无需在每个 handler 中手动添加日志代码。

四. 数据、结果分析

任务一：简单 HTTP 服务器

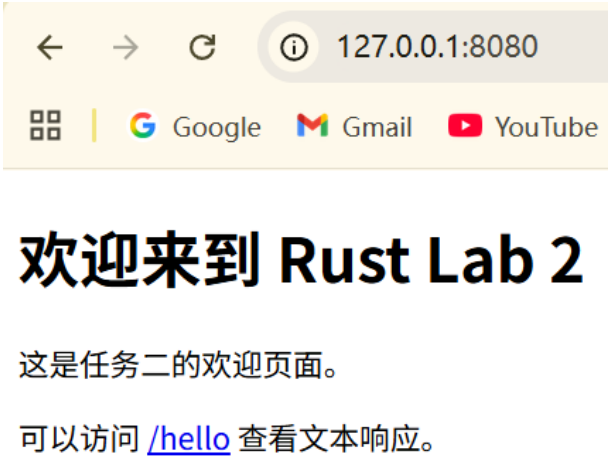
```
   Compiling zstd v0.13.3
   Compiling ryu v1.0.23
   Compiling httpdate v1.0.3
   Compiling base64 v0.22.1
   Compiling language-tags v0.3.2
   Compiling foldhash v0.1.5
   Compiling serde_urlencoded v0.7.1
   Compiling actix-server v2.6.0
   Compiling actix-http v3.12.1
   Compiling cookie v0.16.2
   Compiling serde_json v1.0.149
   Compiling url v2.5.8
   Compiling actix-web-codegen v4.3.0
   Compiling lazy_static v1.5.0
   Compiling impl-more v0.1.9
   Compiling sharded-slab v0.1.7
   Compiling v_htmlescape v0.15.8
   Compiling matchers v0.2.0
   Compiling tracing-log v0.2.0
   Compiling thread_local v1.1.9
   Compiling nu-ansi-term v0.50.3
   Compiling http-range v0.1.5
   Compiling actix-web v4.13.0
   Compiling tracing-subscriber v0.3.23
   Compiling actix-files v0.6.10
   Compiling web_lab v0.1.0 (/home/wuji/home/rustlab/lab2/web_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 17.74s
     Running `target/debug/web_lab`
Server running at http://127.0.0.1:8080
```



Hello, Rust!

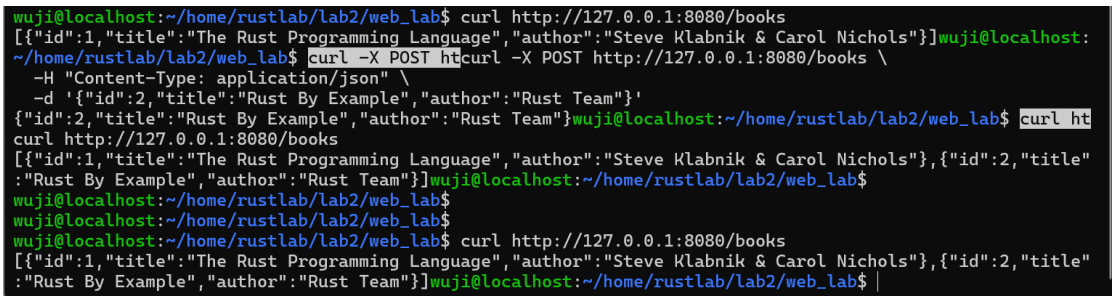
结果分析：程序启动后在终端输出监听地址，使用浏览器访问 http://127.0.0.1:8080/hello，页面正确显示 "Hello, Rust!" 纯文本内容，HTTP 状态码为 200 OK，响应 Content-Type 为 text/plain，符合预期。

任务二：欢迎页面



结果分析：访问根路径 http://127.0.0.1:8080/ 时，浏览器正确渲染了 index.html 中的欢迎页面，包含标题和超链接；点击页面中的 /hello 链接可跳转至任务一的文本响应页面。静态文件托管功能正常，actix-files 能够正确识别 index.html 作为目录索引文件。

任务三：图书管理 API



结果分析：使用 curl 命令测试 API 接口。curl http://127.0.0.1:8080/books 返回 JSON 数组，包含预置的一本书籍数据，格式正确。通过 curl -X POST 添加新书后，再次 GET /books 可以看到列表中包含两本书，返回状态码分别为 200 和 201，数据的序列化与反序列化均正常工作，内存中的共享状态在多次请求间正确保持。

任务四：日志输出

```
wuji@localhost:~/home/rustlab/lab2/web_lab$ curl http://127.0.0.1:8080/
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <title>Rust Lab 2</title>
</head>
<body>
    <h1>欢迎来到 Rust Lab 2</h1>
    <p>这是任务二的欢迎页面。</p>
    <p>可以访问 <a href="/hello">/hello</a> 查看文本响应。</p>
</body>
</html>wuji@localhost:~/home/rustlab/lab2/web_lab$
wuji@localhost:~/home/rustlab/lab2/web_lab$
wuji@localhost:~/home/rustlab/lab2/web_lab$ curl http://127.0.0.1:8080/hello
Hello, Rust!wuji@localhost:~/home/rustlab/lab2/web_lab$
wuji@localhost:~/home/rustlab/lab2/web_lab$ curl http://127.0.0.1:8080/books
[{"id":1,"title":"The Rust Programming Language","author":"Steve Klabnik & Carol Nichols"}]wuji@lo
calhost:~/home/rustlab/lab2/web_lab$
wuji@localhost:~/home/rustlab/lab2/web_lab$ curl -X POST http://127.0.0.1:8080/books \
  -H "Content-Type: application/json" \
  -d '{"id":2,"title":"Rust By Example","author":"Rust Team"}'
{"id":2,"title":"Rust By Example","author":"Rust Team"}wuji@localhost:~/home/rustlab/lab2/web_lab$
```

```
wuji@localhost:~/home/rustlab/lab2/web_lab$ cargo run
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.07s
     Running `target/debug/web_lab`
Server running at http://127.0.0.1:8080
[2026-04-22T07:40:22Z INFO  actix_server::builder] starting 4 workers
[2026-04-22T07:40:22Z INFO  actix_server::server] Actix runtime found; starting in Actix runtime
[2026-04-22T07:40:22Z INFO  actix_server::server] starting service: "actix-web-service-127.0.0.1:8080", wo
rkers: 4, listening on: 127.0.0.1:8080
[2026-04-22T07:40:36Z INFO  actix_web::middleware::logger] GET / HTTP/1.1 200 1.002653 ms
[2026-04-22T07:40:41Z INFO  actix_web::middleware::logger] GET /hello HTTP/1.1 200 0.112556 ms
[2026-04-22T07:40:49Z INFO  actix_web::middleware::logger] GET /books HTTP/1.1 200 0.096417 ms
[2026-04-22T07:40:56Z INFO  actix_web::middleware::logger] POST /books HTTP/1.1 201 0.162170 ms
```

结果分析：添加日志中间件后，服务器终端在每次收到请求时自动输出日志信息，包含请求方法与路径、响应状态码和处理耗时。日志级别为 info，通过 env_logger 输出到标准输出，格式清晰、信息完整。

五. 总结

本次实验以 actix-web 框架为核心，渐进式地完成了从基础 HTTP 服务器到具备 RESTful API 和日志功能的 Web 应用的搭建。四个任务层层递进：任务一建立了最小可运行的异步 HTTP 服务器，理解了 actix-web 的路由注册与异步处理模型；任务二引入 actix-files 实现静态资源托管，了解了路由匹配的优先级机制；任务三通过 serde 和 Mutex<Vec<T>> 实现了带状态的 RESTful API，体会

到 Rust 在并发场景下通过类型系统保证数据安全的设计理念——编译器强制要求使用 Mutex 包裹共享可变数据，从根本上避免了数据竞争；任务四借助 actix-web 内置的 Logger 中间件和 env_logger，以极少的代码量实现了请求级别的日志记录。 总体而言，通过本次实验，我对 Rust 异步编程和 Web 开发有了初步的实践认识，也进一步体会到 Rust 的所有权和类型系统在构建并发安全的服务端应用中的优势。